FYBCA Semester 2 — Complete Exam Notes

Subject 4: Web Design-II (JavaScript)

KBCNMU | Exam: 30 Marks | Complete Notes

TOPIC 1: Website Development Concepts

1.1 What is a Website?

A website is a collection of related web pages hosted on a server and accessible via the internet through a browser. Each page is written in HTML, styled with CSS, and made interactive with JavaScript.

Term	Definition
Web Page	A single document written in HTML displayed in a browser.
Website	A collection of related web pages under one domain.
Web Browser	Software used to view web pages (Chrome, Firefox, Edge).
Web Server	Computer that stores and serves web pages to clients.
URL	Uniform Resource Locator — address of a web page (https://example.com).
HTTP/HTTPS	Protocol for transferring web pages. HTTPS is secure version.
Domain Name	Human-readable address (example.com) that maps to an IP address.

1.2 Client-Side vs Server-Side Scripting

Feature	Client-Side Scripting	Server-Side Scripting
Where executed	User's browser (client)	Web server
Language	JavaScript, HTML, CSS	PHP, Python, Java, ASP.NET
Response time	Faster (no server round-trip)	Slower (request-response)
Security	Less secure (code visible)	More secure (code hidden)
Data access	Cannot access database directly	Can access database
Purpose	UI interaction, validation, animation	Business logic, database, authentication
Example	Form validation, dropdown menus	Login system, e-commerce cart

1.3 Web Technologies Stack

Technology	Role	Example
HTML	Structure/content of web page	Headings, paragraphs, tables
CSS	Styling and layout	Colors, fonts, responsive design

JavaScript	Interactivity and behavior	Form validation, animations, AJAX
PHP/Python	Server-side logic	Processing forms, database queries
MySQL/MongoDB	Database storage	Storing user data

Exam Questions

- What is client-side scripting? Differentiate from server-side scripting. (4 marks)
- What is a website? Explain the role of HTML, CSS, JavaScript. (4 marks)

TOPIC 2: Introduction to JavaScript

2.1 What is JavaScript?

JavaScript (JS) is a lightweight, interpreted, client-side scripting language used to make web pages interactive and dynamic. It was created by Brendan Eich at Netscape in 1995. It runs directly in the web browser.

2.2 Features of JavaScript

- Interpreted Language: Code executes directly in browser, no compilation needed.
- Client-Side: Runs in user's browser, reducing server load.
- Event-Driven: Responds to user actions (click, hover, keypress).
- Object-Based: Uses built-in objects (String, Math, Array, Date).
- Case-Sensitive: 'name' and 'Name' are different.
- Loosely Typed: No need to declare data type of variables.
- Platform Independent: Runs on any OS with a browser.

2.3 Advantages of JavaScript

Advantage	Explanation
Speed	Executes in browser without server communication — very fast.
Simplicity	Easy to learn and implement.
Interoperability	Works with HTML and CSS seamlessly.
Rich Interfaces	Enables drag-drop, sliders, dynamic content.
Reduced Server Load	Validation done on client side — less server requests.
No Extra Software	Just a text editor and browser needed.

2.4 Limitations of JavaScript

Limitation	Explanation
Security Risk	Code is visible in browser — can be exploited.
Browser Dependency	Different browsers may interpret JS differently.
No File Access	Cannot read/write files on client machine (security).
Disabled by User	User can disable JavaScript in browser settings.
Single-threaded	Executes one task at a time (though async helps).

2.5 Embedding JavaScript in HTML

Method 1: Inline JavaScript

```
<button onclick="alert('Hello!')">Click Me</button>
```

Method 2: Internal JavaScript (<script> tag)

```
<!DOCTYPE html>
<html>
<head>
    <title>My Page</title>
    <script>
        function greet() {
            alert("Hello World!");
        }
    </script>
</head>
<body>
    <button onclick="greet()">Click</button>
</body>
</html>
```

Method 3: External JavaScript File

```
<!-- In HTML file -->
<script src="script.js"></script>

// In script.js file:
function greet() {
    alert("Hello from external file!");
}
```

Method	Where Written	Advantage
Inline	Inside HTML tag attribute	Quick for small code
Internal	<script> tag inside HTML	Good for single page
External	.js file linked with src=	Best: reusable, maintainable

Exam Questions

- What is JavaScript? What are its features? (4 marks)
- What are the advantages and limitations of JavaScript? (4 marks)
- How can JavaScript be embedded in HTML? Explain all methods. (6 marks)

TOPIC 3: Data Types and Variables

3.1 Variables

Variables are named containers that store data values. In JavaScript, variables are declared using var, let, or const.

```
var name = "Rahul";       // Old way (function scope)
let age = 20;             // Modern way (block scope)
const PI = 3.14159;       // Constant (cannot be changed)

name = "Vishal";  // Can change var and let
// PI = 3;        // ERROR: Cannot reassign const
```

Keyword	Scope	Reassignable?	Redeclarable?
var	Function scope	Yes	Yes
let	Block scope	Yes	No
const	Block scope	No	No

3.2 Data Types in JavaScript

Primitive Data Types

Data Type	Description	Example
Number	Integer or floating-point numbers	let x = 42; let y = 3.14;
String	Sequence of characters in quotes	let s = "Hello"; let t = 'World';
Boolean	True or false value only	let flag = true;
Undefined	Variable declared but not assigned	let x; // x is undefined
Null	Intentionally empty value	let x = null;
Symbol	Unique identifier (ES6)	Symbol('id')

Non-Primitive Data Types

Type	Description	Example
Object	Collection of key-value pairs	let obj = {name:'Rahul', age:20};
Array	Ordered list of values	let arr = [1, 2, 3, 'hello'];
Function	Block of reusable code	function add(a,b) { return a+b; }

3.3 typeof Operator

```
typeof 42            // "number"
typeof "hello"       // "string"
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof null          // "object" (quirk of JS!)
typeof {}            // "object"
typeof []            // "object"
typeof function(){}  // "function"
```

TOPIC 4: Operators and Expressions

4.1 Arithmetic Operators

Operator	Symbol	Example	Result
Addition	+	5 + 3	8
Subtraction	-	10 - 4	6
Multiplication	*	3 * 4	12
Division	/	15 / 4	3.75
Modulus (remainder)	%	10 % 3	1
Increment	++	x=5; x++	x becomes 6
Decrement	--	x=5; x--	x becomes 4
Exponentiation	**	2 ** 3	8

4.2 Comparison Operators

Operator	Meaning	Example	Result
==	Equal to (value only)	5 == '5'	true
===	Strictly equal (value + type)	5 === '5'	false
!=	Not equal to	5 != 4	true
!==	Strictly not equal	5 !== '5'	true
>	Greater than	8 > 5	true
<	Less than	3 < 7	true
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	4 <= 3	false

4.3 Logical Operators

Operator	Symbol	Meaning	Example
AND	&&	True only if BOTH are true	(5>3) && (2<4) => true
OR	||	True if AT LEAST ONE is true	(5>3) || (2>4) => true
NOT	!	Reverses the boolean value	!(5>3) => false

4.4 Assignment Operators

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 2	x = x - 2
*=	x *= 4	x = x * 4

/=	x /= 2	x = x / 2
%=	x %= 3	x = x % 3

4.5 String Operator (+) — Concatenation

```
let firstName = "John";
let lastName = "Doe";
let fullName = firstName + " " + lastName;
console.log(fullName);   // Output: John Doe

// Number + String = String!
console.log(5 + "5");    // Output: "55" (concatenation)
console.log(5 + 5 + "5"); // Output: "105"
```

TOPIC 5: Conditional Statements

5.1 if Statement

```
let age = 18;
if (age >= 18) {
    console.log("You are an adult.");
}
```

5.2 if-else Statement

```
let marks = 45;
if (marks >= 50) {
    console.log("Pass");
} else {
    console.log("Fail");
}
```

5.3 if-else if-else (Ladder)

```
let marks = 75;

if (marks >= 75) {
    console.log("Distinction");
} else if (marks >= 60) {
    console.log("First Class");
} else if (marks >= 50) {
    console.log("Second Class");
} else if (marks >= 40) {
    console.log("Pass");
} else {
    console.log("Fail");
}
```

5.4 switch Statement

```
let day = 3;

switch(day) {
    case 1: console.log("Monday"); break;
    case 2: console.log("Tuesday"); break;
    case 3: console.log("Wednesday"); break;
    case 4: console.log("Thursday"); break;
    case 5: console.log("Friday"); break;
    default: console.log("Weekend");
}
```

IMPORTANT: Always use 'break' after each case to prevent fall-through.

if-else	switch
Tests range or complex conditions	Tests specific value of a variable
Can test different variables	Tests same variable against multiple values
More flexible	Cleaner for multiple specific values
Works with any data type	Works best with numbers and strings

TOPIC 6: Looping Statements

6.1 for Loop

```
// Syntax: for(initialization; condition; increment)
for (let i = 1; i <= 5; i++) {
    console.log("Number: " + i);
}
// Output: Number: 1, 2, 3, 4, 5
```

6.2 while Loop

```
let i = 1;
while (i <= 5) {
    console.log("Count: " + i);
    i++;
}
// Condition checked BEFORE loop body executes
```

6.3 do-while Loop

```
let i = 1;
do {
    console.log("Number: " + i);
    i++;
} while (i <= 5);
// Loop body executes AT LEAST ONCE (condition checked AFTER)
```

Feature	for	while	do-while
Use when	Iterations known	Condition-based	Must execute at least once
Condition check	Before loop	Before loop	After loop body
Minimum executions	0	0	At least 1
Counter/init	In loop header	Before loop	Before loop

6.4 break and continue

```
// break: exits loop immediately
for (let i = 1; i <= 10; i++) {
    if (i == 5) break;     // Stops loop when i=5
    console.log(i);        // Prints 1 2 3 4
}

// continue: skips current iteration
for (let i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;  // Skip even numbers
    console.log(i);            // Prints 1 3 5 7 9
}
```

TOPIC 7: Functions in JavaScript

7.1 What is a Function?

A function is a reusable block of code that performs a specific task. It is defined once and can be called multiple times. Functions make code modular, readable, and maintainable.

7.2 Types of Functions

1. Named Function

```
function greet(name) {
    return "Hello, " + name + "!";
}
console.log(greet("Rahul"));    // Output: Hello, Rahul!
```

2. Function Expression

```
const add = function(a, b) {
    return a + b;
};
console.log(add(3, 4));    // Output: 7
```

3. Arrow Function (ES6)

```
const multiply = (a, b) => a * b;
console.log(multiply(3, 4));    // Output: 12
```

4. Function with Default Parameters

```
function greet(name = "Guest") {
    return "Hello, " + name;
}
console.log(greet());           // Hello, Guest
console.log(greet("Vishal")); // Hello, Vishal
```

5. Recursive Function

```
function factorial(n) {
    if (n <= 1) return 1;           // Base case
    return n * factorial(n - 1);    // Recursive call
}
console.log(factorial(5));    // 5*4*3*2*1 = 120
```

TOPIC 8: Built-in String Functions

8.1 Common String Methods

Method	Description	Example
length	Returns length of string	'hello'.length => 5
toUpperCase()	Converts to UPPERCASE	'hello'.toUpperCase() => 'HELLO'
toLowerCase()	Converts to lowercase	'HELLO'.toLowerCase() => 'hello'
charAt(i)	Returns character at index i	'hello'.charAt(1) => 'e'
indexOf(str)	Returns index of first occurrence (-1 if not found)	'hello'.indexOf('l') => 2
lastIndexOf(str)	Returns index of last occurrence	'hello'.lastIndexOf('l') => 3
substring(s,e)	Extracts characters from s to e-1	'hello'.substring(1,3) => 'el'
slice(s,e)	Like substring but allows negative indices	'hello'.slice(-3) => 'llo'
replace(old,new)	Replaces first occurrence	'hello'.replace('l','r') => 'herlo'
split(sep)	Splits string into array	'a,b,c'.split(',') => ['a','b','c']
trim()	Removes whitespace from both ends	' hi '.trim() => 'hi'
includes(str)	Returns true if string contains str	'hello'.includes('ell') => true
startsWith(str)	Returns true if starts with str	'hello'.startsWith('he') => true
endsWith(str)	Returns true if ends with str	'hello'.endsWith('lo') => true
concat(str)	Joins two strings	'Hello'.concat(' World') => 'Hello World'

TOPIC 9: Dialog Boxes and Events

9.1 Dialog Boxes

Dialog	Syntax	Description	Returns
Alert	alert('message')	Shows message, only OK button.	undefined
Confirm	confirm('message')	Shows message with OK and Cancel.	true/false
Prompt	prompt('message', 'default')	Shows message with input field.	String or null

```
// Alert
alert("Welcome to our website!");

// Confirm
let result = confirm("Are you sure you want to delete?");
if (result == true) {
    console.log("Deleted!");
} else {
    console.log("Cancelled.");
}

// Prompt
let name = prompt("What is your name?", "Guest");
console.log("Hello, " + name);
```

9.2 JavaScript Events

Events are actions that happen in the browser (user clicks, hovers, types). JavaScript responds to these events using EVENT HANDLERS.

Event	When it Fires	HTML Usage
onclick	User clicks an element	<button onclick='func()'>
ondblclick	User double-clicks	
onmouseover	Mouse pointer moves over element	<img onmouseover='func()'>
onmouseout	Mouse pointer leaves element	
onkeypress	User presses a key	<input onkeypress='func()'>
onkeyup	User releases a key	<input onkeyup='func()'>
onkeydown	User presses down a key	<input onkeydown='func()'>
onchange	Value of element changes	<select onchange='func()'>
onsubmit	Form is submitted	<form onsubmit='func()'>
onload	Page finishes loading	<body onload='func()'>
onfocus	Element gets focus	<input onfocus='func()'>
onblur	Element loses focus	<input onblur='func()'>

```
// Example: Event handling
<button onclick="changeColor()">Change Color</button>
```

```
<script>
function changeColor() {
    document.body.style.backgroundColor = "lightblue";
    alert("Color changed!");
}
</script>
```

Exam Questions

- What are dialog boxes in JavaScript? Explain alert, confirm, prompt. (6 marks)
- What are JavaScript events? List and explain common events. (6 marks)
- Write a JavaScript program to validate a form using events. (6 marks)

TOPIC 10: JavaScript Objects

10.1 String Object

```
var str = new String("Hello World");
console.log(str.length);           // 11
console.log(str.toUpperCase());    // HELLO WORLD
console.log(str.indexOf("World")); // 6
```

10.2 Math Object

The Math object provides mathematical functions and constants. It is NOT a constructor — use directly as Math.method().

Property/Method	Description	Example
Math.PI	Value of pi (3.14159...)	Math.PI => 3.14159
Math.E	Euler's number	Math.E => 2.718
Math.round(x)	Rounds to nearest integer	Math.round(4.6) => 5
Math.floor(x)	Rounds DOWN	Math.floor(4.9) => 4
Math.ceil(x)	Rounds UP	Math.ceil(4.1) => 5
Math.sqrt(x)	Square root	Math.sqrt(16) => 4
Math.pow(x,y)	x raised to power y	Math.pow(2,10) => 1024
Math.abs(x)	Absolute value	Math.abs(-7) => 7
Math.max(...)	Maximum value	Math.max(3,1,7,2) => 7
Math.min(...)	Minimum value	Math.min(3,1,7,2) => 1
Math.random()	Random number 0 to 1	Math.random() => 0.743
Math.log(x)	Natural logarithm	Math.log(1) => 0

10.3 Form Object

The Form object allows JavaScript to access and validate HTML form elements.

```
<form name="myForm" onsubmit="return validateForm()">
    Name: <input type="text" name="uname" /><br>
    Email: <input type="text" name="email" /><br>
    <input type="submit" value="Submit" />
</form>

<script>
function validateForm() {
    var name = document.myForm.uname.value;
    var email = document.myForm.email.value;

    if (name == "") {
        alert("Name is required!");
        return false;   // Prevents form submission
```

```
    }
    if (email == "") {
        alert("Email is required!");
        return false;
    }
    return true;  // Allow submission
}
</script>
```

Must Remember

- JavaScript is case-sensitive.
- Use === (strict equality) instead of == to avoid type coercion issues.
- var is function-scoped; let and const are block-scoped.
- Math.random() returns value between 0 (inclusive) and 1 (exclusive).
- String index starts from 0.
- typeof null returns 'object' (known JS quirk).
- return false in onsubmit prevents form submission.

QUICK REVISION SUMMARY — Subject 4: JavaScript

Topic	Key Points
JS Introduction	Interpreted, client-side, event-driven. Created 1995 by Brendan Eich.
Embedding	Inline, Internal (<script>), External (.js file). External is best practice.
Variables	var (function scope), let (block scope), const (constant).
Data Types	Number, String, Boolean, Undefined, Null, Object, Array.
Operators	Arithmetic, Comparison (==/===), Logical (&&/||/!), Assignment.
if-else	if, else if, else. switch for specific value comparison.
Loops	for, while, do-while. break exits, continue skips.
Functions	Named, Expression, Arrow, Default params, Recursive.
String Methods	length, toUpperCase, indexOf, substring, replace, split, trim.
Dialog Boxes	alert (message), confirm (true/false), prompt (string input).
Events	onclick, onchange, onsubmit, onload, onkeyup, onmouseover, onfocus.
Math Object	Math.PI, round(), floor(), ceil(), sqrt(), pow(), random(), max(), min().
Form Validation	Access: document.formName.fieldName.value. return false prevents submit.